



List scheduling with duplication for heterogeneous computing systems

Xiaoyong Tang^{a,b}, Kenli Li^{a,*}, Guiping Liao^b, Renfa Li^a

^a School of Computer and Communication, Hunan University, Changsha, 410082, China

^b Information Science and Technology College, Hunan Agricultural University, Changsha, 410128, China

ARTICLE INFO

Article history:

Received 20 November 2008

Received in revised form

22 April 2009

Accepted 19 January 2010

Available online 1 February 2010

Keywords:

List scheduling

Heterogeneous computing systems

DAG

Duplication

ABSTRACT

Effective task scheduling is essential for obtaining high performance in heterogeneous computing systems (HCS). However, finding an effective task schedule in HCS, requires the consideration of the heterogeneity of computation and communication. To solve this problem, we present a list scheduling algorithm, called Heterogeneous Earliest Finish with Duplicator (HEFD). As task priority is a key attribute for list scheduling algorithm, this paper presents a new approach for computing their priority which considers the performance difference in target HCS using variance. Another novel idea proposed in this paper is to try to duplicate all parent tasks and get an optimal scheduling solution. The comparison study, based on both randomly generated graphs and the graphs of some real applications, shows that our scheduling algorithm HEFD significantly surpasses other three well-known algorithms.

© 2010 Elsevier Inc. All rights reserved.

1. Introduction

Recently there has been a great interest in heterogeneous distributed computing systems, mainly due to the low cost of their resources and high cost of mainframes. Heterogeneous systems are usually composed of diverse sets of resources interconnected with high-speed networks. In opposition to mainframes, heterogeneous computing systems (HCS) are more adequate for executing computationally intensive parallel and distributed applications. However, to fully exploit and effectively use any HCS, a resource management software must be provided to mask the complexity of different physical architectures for the user. This complexity arises in managing communication, synchronization and scheduling a large number of tasks, in dealing with portability of library facilities used to parallelize and/or distribute user applications, and in many other related issues [24]. More and more evidence shows that the scheduling of workflow applications is highly critical to the performance of heterogeneous computing systems. It deals with the allocation of individual tasks to suitable processors and assignment of the proper order of task execution on each resource. The common objective of scheduling is to map tasks onto machines and order their execution so that task precedence requirements are satisfied and there is a minimum schedule length (makespan) [18,19,13].

A popular representation of a parallel application is the directed acyclic graph (DAG) in which the nodes represent application

tasks and the directed arcs or edges represent inter-task data dependencies, such as task's precedence. The problem of finding the optimal schedule is NP-complete [14] in general. Therefore, heuristics can be used to obtain a sub-optimal scheduling rather than parsing all possible schedules. Task scheduling has been extensively studied, and various heuristics have been proposed in the literature [10,31,25,23,17,20,28,29,12,16,21,32,7,30,27,1,8,22,9,2,11,3,15,4,34,5,26]. In compile-time scheduling, these heuristics are classified into a variety of categories (such as list scheduling, clustering and task duplication).

The basic idea of list scheduling is to assign priorities to the tasks of the DAG and place the tasks in a list arranged in descending order of priorities. A task with a higher priority is scheduled before a task with a lower priority and ties are broken using some method, such as its child's priority. List scheduling is generally accepted as an attractive approach since it combines low complexity with good results [10,31,25,23,17,20,28,29,12,16,21,32,7,30,27]. The basic idea of task duplication is to try to duplicate the parents of the current selected task onto the selected machine or onto another machines, aiming to reduce or optimize the task finish time [1,8,22,9,2,11]. The main weakness of duplication-based algorithms is their high complexity and that they mainly target an unbounded number of computing machines.

The recently reported SD algorithm [3] blends the simplicity of list-based approaches with the performance of duplication-based techniques, and is shown to be quite effective for homogeneous computing systems. It improves the performance of list scheduling algorithms by incorporating limited yet effective duplication into it. Idle time slots, left in a processor, are exploited for duplicating immediate parents of a child node so as to reduce its waiting

* Corresponding author.

E-mail addresses: tang_313@163.com (X. Tang), lk1510@263.net (K. Li), lgpxf@hunau.net (G. Liao), scc_lrf@hnu.cn (R. Li).

time on the processor. Some techniques, such as HCPFD [15], HLD [4], were introduced in heterogeneous distributed systems. The comparison study in [15,4] shows that HCPFD, HLD significantly surpass other algorithms, such as DLS [27], HEFT [21,32], LDBS [11]. However, the duplication of immediate parents was limited by critical path, and their idle time slots were not adequately exploited.

In HCS, the various computation costs of the same task on different processors present us with a problem: the priorities computed using the computation costs of tasks on a particular processor may differ from the priorities computed using the same computation costs of tasks on another processor. To overcome this problem, previous scheduling algorithms for HCS set the computation costs of tasks to their median values, as in the HCPFD, DLS algorithm, or their mean values, as in the HEFT, HLD algorithm, in order to get a single computation cost for each task. However, these techniques estimate approximate computation costs of tasks and hence limit the ability of scheduling algorithms to precisely compute the priorities of tasks. H. Zhao and R. Sakellariou compared different methods such as mean value, median value, best value, and worst value in literature [34]. Results show that significant variations in the makespan underline the dependency of the scheduling algorithms on the weight computing methods. The sensitivity is largely due to the fact that scheduling algorithms have difficulty to access the relative importance of independent tasks effectively.

In this paper, we propose a novel scheduling algorithm based on both list-based approach and duplication-based techniques for heterogeneous distributed computing systems. We suggest a new approach of setting task nodes and edges' weight, which considers the performance difference in target HCS using variance. The duplication of tasks in our algorithm is optimized for all immediate parent tasks. The rest of the article is organized as follows: in the next section, we define the task scheduling problem. Section 3 proposes a method of computing the weight of task nodes and edges by considering heterogeneity of HCS, and some scheduling attributes are also introduced in this section. The heterogeneous earliest finish with duplicator (HEFD) algorithm is proposed in Section 4. In Section 5, the simulation experimental results are presented and analyzed. The article concludes with Section 6.

2. Task scheduling problem

A scheduling system model consists of an application, a target computing environment, and performance criteria for scheduling. The application and computing environment can be represented by a task graph and resource graph respectively. The performance criteria will be discussed in Section 5.

2.1. Parallel application

Generally, an application is represented by a directed acyclic graph $G = \langle V, E \rangle$, where V is the set of v tasks that can be executed on any of the available processors; $E \subseteq V \times V$ is the set of directed arcs or edges between the tasks representing the dependency between the tasks. For example, edge $e_{i,j} \in E$ represents the precedence constraint such that task v_i should complete its execution before task v_j completes its execution. A task may have one or more inputs. When all its inputs are available, the task is triggered to execute. After its execution, it generates its outputs. The weight $w(v_i)$ assigned to node v_i represents its computation cost and the weight $w(e_{i,j})$ assigned to edge $e_{i,j}$ represents its communication cost. We will introduce a new approach to assign weight of task nodes and edges in Section 3. Fig. 1 shows an example DAG with the assigned node and edge weights.

A task with no parent node in the DAG is called an *entry task*, and a task with no child node in the DAG is called an *exit task*. We assume that the DAG has exactly one entry task v_{entry} and one exit task v_{exit} . If multiple exit tasks or entry tasks exist, they may

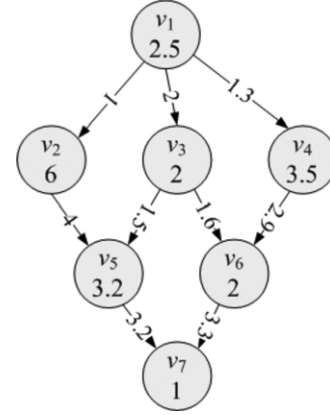


Fig. 1. A parallel application task graph.

be connected with zero time-weight edges to a single pseudo-exit task or a single entry task that has zero time-weight.

2.2. The target system

The topology of the target system is modeled as an undirected graph $GT = \langle P, L \rangle$, P is a finite set of p vertices, and L is a finite set of fully connected undirected edges or arcs. A vertex p_i represents the processor i , and an undirected edge $l_{i,j}$ represents a bidirectional communication link between the incident processors p_i and p_j . The resource graph is a complete graph with p fully connected nodes. A weight $w(p_i)$ assigned to a processor p_i represents the processor computation capacity (the amount of computation that can be performed in a unit time). Similarly, the weight $w(l_{i,j})$ assigned to a link $l_{i,j}$ represents stands for its communication capacity (the amount of data that can go through the link in a unit time). We further assume that all interprocessor communications are performed without contention, and the communication overhead between two tasks scheduled on the same processor is taken as zero. This assumption holds since our computing environment consists of processors connected with wide area network links as pointed out in literature [5].

3. HCS list scheduling

3.1. Setting task node and edge weight

There are various ways to set the weights of task nodes and edges in HCS [34,26]. For instance, one can take average value, median value, best value, etc. In this paper, we consider the effect of heterogeneity of HCS (both computation capacity and communication capacity) in addition to average computation cost and average communication cost. The computation capacity heterogeneity factor α can be computed as follows:

$$\alpha = \frac{\sqrt{\sum_{i=1}^p (w(p_i) - \overline{w(p)})^2}}{p \times \max_{x \leq p} \{w(p_x)\}} \quad (1)$$

where $\overline{w(p)}$ is processor average computation capacity. A high α value indicates high variance of the computation capacity in HCS, and vice versa. If the heterogeneity factor α is set to 0, the computation capacity in HCS is the same for all processors. Similarly, the communication capacity heterogeneity factor β is defined as:

$$\overline{w(l)} = \frac{\sum_{i=1}^p \sum_{j=i+1}^p w(l_{i,j})}{p(p-1)/2} \quad (2)$$

$$\beta = \frac{\sqrt{\sum_{i=1}^p \sum_{j=i+1}^p (w(l_{i,j}) - \overline{w(l)})^2}}{p(p-1)/2 \times \max_{x,y \leq p} \{w(l_{x,y})\}} \quad (3)$$

Thus, the weight of task nodes w_i and edges $d_{i,j}$ are defined as follows:

$$w_i = \frac{w(v_i)}{\alpha \times \min_{x \leq p} \{w(p_x)\} + (1 - \alpha) \times \max_{x \leq p} \{w(p_x)\}} \quad (4)$$

$$d_{i,j} = \frac{w(e_{i,j})}{\beta \times \min_{x,y \leq p} \{w(l_{x,y})\} + (1 - \beta) \times \max_{x,y \leq p} \{w(l_{x,y})\}} \quad (5)$$

3.2. Some scheduling attributes

Before presenting the objective function, it is necessary to define the EST and EFT attributes, which are derived from a given partial schedule. $EST(v_i, p_m)$ and $EFT(v_i, p_m)$ are the earliest execution start time and the earliest execution finish time of task v_i on processor p_m , respectively. For the entry task v_{entry} , the EST is defined by

$$EST(v_{entry}, p_m) = 0. \quad (6)$$

For the other tasks in the graph, the EST (Earliest Start Time) and EFT (Earliest Finish Time) values are computed recursively, starting from the entry task, as shown in (7) and (8), respectively. In order to compute the EFT for a task v_i , all immediate predecessor tasks of v_i must have been scheduled.

$$EST(v_i, p_m) = \max\{Available(v_i, p_m), \max_{v_j \in pred(v_i)} \{EFT(v_j, p_n) + c_{j,i}\}\} \quad (7)$$

$$EFT(v_i, p_m) = EST(v_i, p_m) + w(v_i)/w(p_m) \quad (8)$$

$$c_{j,i} = w(e_{j,i})/w(l_{n,m}) \quad (9)$$

where $c_{j,i}$ is the communication cost of edge $e_{j,i}$ transferring data from task v_j (scheduled on p_n) to task v_i (scheduled on p_m), which is computed by Eq. (9). When both v_j and v_i are scheduled on the same processor, $c_{j,i}$ becomes zero, since we assume that the intraprocessor communication cost is negligible when it is compared with the interprocessor communication cost. The $pred(v_i)$ is the set of immediate predecessor tasks to task v_i , and $Available(v_i, p_m)$ is the earliest time at which processor p_m is ready for task execution. Using insertion-based policy [22], the time between $EST(v_i, p_m)$ and $EFT(v_i, p_m)$ is also available. The inner earliest block in the EST equation returns the ready time, i.e., the time when all data needed by v_i has arrived at processor p_m . After all tasks in a graph are scheduled, the schedule length (i.e., overall completion time) will be the actual finish time of the exit task v_{exit} , thus the schedule length (which is also called makespan) is defined as follows.

$$makespan = EFT(v_{exit}). \quad (10)$$

The objective function of the task scheduling problem is to determine the assignment of tasks of a given application to processors such that its scheduling length is minimized.

4. Proposed algorithm

This section presents an algorithm for list scheduling on a bounded number of fully connected heterogeneous computing systems called Heterogeneous Earliest Finish with Duplicator (HEFD), which aims to achieve high performance and low complexity. The algorithm consists of two mechanisms, a listing mechanism, which is a modified version of the HEFT [21,32] heuristic for heterogeneous computing systems and a machine assignment mechanism

which is a near lower-bound complexity mechanism based on task duplication. The pseudo-code of the algorithm is shown in Algorithm 1. The algorithm completes in two main phases as described in the following sections.

4.1. Task priority

Tasks are ordered in our algorithm by their scheduling priorities based on ranking [21,32]. The rank of task v_i is recursively defined by

$$rank(v_i) = w_i + \max_{v_j \in succ(v_i)} \{d_{i,j} + rank(v_j)\} \quad (11)$$

where $succ(v_i)$ is the set of immediate successors of task v_i , w_i is the computation weight of task v_i which is defined in (4) and $d_{i,j}$ is the communication weight of edge $e_{i,j}$ which is also defined in (5). The rank is computed recursively by traversing the task graph upward, starting from the exit task. For the exit task v_{exit} , the rank value is equal to

$$rank(v_{exit}) = w_{exit}. \quad (12)$$

Basically, $rank(v_i)$ is the length of the critical path from task v_i to the exit task, including the computation weight of task v_i .

```

1 Set the weight of task nodes and communication edges use
  formula (4),(5) respectively
2 Compute rank value for all tasks by traversing application
  graph, starting from the exit task
3 Sort the tasks in a scheduling list by non-increasing order of
  rank value
4 while there are unscheduled tasks in the list do
5   Select the first task  $v_i$  from the list for scheduling
6   for each processor  $p_k$  in the processor set ( $p_k \in P$ ) do
7     Compute the data arrival time use Eq.(13)
8     Sort immediate parents  $v_j$  by non-increasing order of
      data arrival time
9     while there are immediate parents  $v_j$  not duplicated do
10      if the duplication of  $v_j$  can reduce its start execution
        time then
11        duplicate  $v_j$ 
12      end
13    end
14    Compute the earliest finish time by Eq.(8)
15  end
16  find the minimum earliest finish time processor  $p_k$ 
17  Assign task  $v_i$  to processor  $p_k$  with minimize  $EFT(v_i, p_k)$ 
18 end

```

Algorithm 1: The pseudo-code for HEFD algorithm

4.2. Task assignment and duplication phase

In this phase, tasks are assigned to the processors and duplication is employed to reduce their finish time. First unscheduled task in the task sequence is selected and scheduled on a processor that can complete its execution at the earliest using duplication. It is to be noted that this processor may or may not be its best-suited processor. This type of heuristic automatically takes into consideration the computing power of the candidate processor. Free time slots, if any, in the processor's schedule list are exploited in this process to accommodate the candidate node and duplicated nodes to facilitate their early start as explained below.

To start execution on a processor p_k , the task v_i has to wait for the data arrivals from all of its immediate parents ($v_j \in pred(v_i)$), and v_j execute on processor p_n . Data arrival time for v_i on p_k from v_j is given by

$$DAT(v_i, p_k, v_j)_{v_j \in pred(v_i)} = EFT(v_j, p_n) + c_{j,i}. \quad (13)$$

Due to the non-availability of data earlier, owing to precedence-constraints in task graph or communication delays in the processor network, a processor may remain idle leading to the formation of scheduling holes. The algorithm exploits suitable scheduling holes as available on p_k for replicating immediate parents which can reduce its start execution time, so as to improve its data arrival time. Suitability of a free slot to accommodate or duplicate a task is detail discussed in literature [3].

4.3. The time-complexity analysis

The time complexity of scheduling algorithms for DAG is usually expressed in terms of number of node v , number of edges e , and number of processors p . The time complexity of HEFD is analyzed as follows.

Computing the weight of task nodes and communication edges can be done in time $O(vp)$ and $O(ep^2)$, respectively. Computing the rank value and sorting the tasks can be done in time $O(v \log v)$. The processor selection for all tasks can be done in time $O(v|v|^2p)$, and $|v|$ is the max degree of v_i in application DAG. As $|v|^2$ is approximate the number of edges e , and thus the time complexity is $O(vep)$.

5. Simulation experiment results

In this section, first, we compare performance of HEFT [21,32] and new weight HEFT. Then, we compare the performance of the HEFD algorithm to three well-known scheduling algorithms in HCS: the HEFT [21,32], HLD [4], and HCPFD [15] algorithms. To test the performance of the scheduling algorithms, an extensive simulation environment of HCS with 64 nodes, is built and run on varying from Pentium II to Pentium IV, based on Hunan University Supercomputer Center (HUSC). In our simulation experiments, we randomly choose 32 nodes which satisfies with five different computation capacity heterogeneity factor α values: 0.1, 0.2, 0.4, 0.6, and 0.8.

In this paper, we consider two sets of graphs as the workflow applications for testing the algorithms: randomly generated application graphs and graphs that represent some of the numerical real-world problems. The comparison is intended not only to present quantitative results, but also to qualitatively analyze the results and to suggest explanations, for a better insight in the overall scheduling problem.

The main performance metric chosen for the comparison is the makespan (see Eq. (10)). Since a large set of application graphs with different properties is used, it is necessary to normalize the schedule length to the lower bound, which is called normalized schedule length (NSL). The principal performance metric of an algorithm is the length of its output schedule. The NSL of an algorithm is defined as:

$$NSL = \frac{\text{makespan}}{\sum_{v_i \in CP_{min}, p_m \in P} \min\{w(v_i)/w(p_m)\}}. \quad (14)$$

CP_{min} is the critical path of the DAG when the task node weights are evaluated as the minimum computation cost among all capable processors. The denominator represents a lower bound on the schedule length. Such a lower bound may not always be possible to reach and $NSL \geq 1$ for any algorithm. We use averaged NSL over a set of DAGs as a comparison metric.

5.1. Randomly generated application graphs

For the generation of random graphs, which are commonly used to compare scheduling algorithms [32,7,30], three fundamental characteristics of the DAG are considered:

- *DAG size, n* : The number of tasks in the application DAG.
- *Communication to computation cost ratio, CCR*: The average communication cost divided by the average computation cost of the application DAG.
- *Parallelism factor, λ* : The number of levels of the application DAG is calculated by randomly generating a number, using a uniform distribution with a mean value of $\frac{\sqrt{n}}{\lambda}$, and then rounding it up to the nearest integer. The width of each level is calculated by randomly generating a number using a uniform distribution with a mean value of $\lambda \times \sqrt{n}$ and then rounding it up to the nearest integer [32,7]. A low λ value leads to a DAG with a low parallelism degree [7,3].

In our simulation experiments, Graphs were generated for all combinations of the above parameters with the number of nodes ranging between 400 and 2000, in steps of 400. Every possible edge (DAGs are acyclic) was created with the same probability, which was calculated based on the average number of edges per node. To obtain the desired CCR for a graph, node weights are taken randomly from a uniform distribution [0.1, 1.9] around 1, thus the average node weight is 1. Edge weights are also taken from a uniform distribution, whose mean depends on CCR (0.1, 0.5, 1.0, 2.0 and 5.0) and *Parallelism factor* λ (0.2, 0.5, 1.0, 2.0 and 5.0). The relative deviation of the edge weights is identical to that of the node weights. Every set of the above parameters was used to generate several random graphs in order to avoid scattering effects. The results presented below are the average of the results obtained for these graphs.

5.2. HEFT and New Weight HEFT performance results

In this experiments, we investigate how the weight assignment method will impact the average NSL. The results are shown in Fig. 2. Two algorithms, namely HEFT [21,32] and New Weight HEFT are compared. The New Weight HEFT is adapted from HEFT where our weight assignment method described in Section 3.1 is adopted. We only show the cases where $CCR = 0.1, 1$, and 10 due to space limits. From Fig. 2 we first observe that for all cases the average NSLs show an increasing trend with respect to the increasing of task graph size. This is due to the fact that the proportion of task nodes other than those on the critical path increases with the task graph size, making it more difficult to achieve the lower bound. When $CCR = 0.1$, the improvement of New Weight HEFT over HEFT is 2.06% when task size is 80. As the task size reach 200, the improvement is reach 6.25%. When CCR increases to 10, the improvements are also similar to $CCR = 0.1$.

5.3. Random application performance results

The performances of the algorithms were compared with respect to various graph characteristics. The first set of experiments compares the performance and cost of the algorithms with respect to various graph sizes (see Fig. 3). The makespan value of the HEFD algorithm is shorter than the HEFT, HLD and HCPFD algorithms by: (0.82%, 0.41%, 0.42%), (1.82%, 0.91%, 0.82%), (1.72%, 1.11%, 1.42%), (2.34%, 1.41%, 0.99%) and (3.5%, 2.1%, 2.2%) for DAG size of: 400, 800, 1200, 1600 and 2000, respectively. The first value of each parenthesized pair is the improvement achieved by the HEFD algorithm over the HEFT algorithm, while the second value is the improvement of the HEFD over the HLD algorithm, the other value is the improvement of the HEFD over the HCPFD algorithm. The NSL values achieved by the four algorithms with respect to certain DAG size are shown in Fig. 3(b). The average NSL values are also shorter than HEFT, HLD, and HCPFD, when the DAG size is equal to: 400, 800, 1200, 1600, and 2000, respectively. In these experiments,

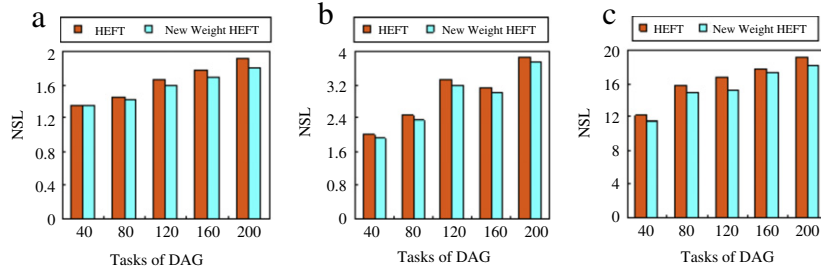


Fig. 2. Performance comparison of weight assignment method. (a) CCR = 0.1; (b) CCR = 1; (c) CCR = 10.

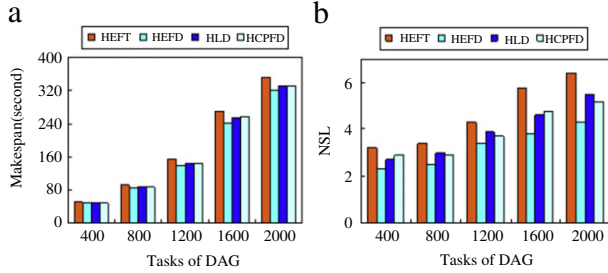


Fig. 3. Performance impact of tasks of DAG: (a) makespan in seconds; (b) NSL.

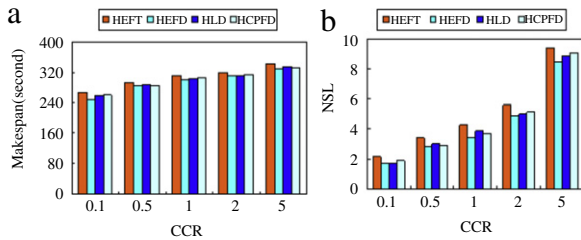


Fig. 4. Performance impact of CCR: (a) makespan in seconds; (b) NSL.

the HEFD algorithm outperforms the HEFT, HLD, and HCPFD algorithms for all tested DAG size values in terms of both makespan and NSL.

The second set of experiments compares the performance of the algorithms with respect to various CCR. The results are as shown in Fig. 4. As the value of CCR increases, interprocessor communication overhead dominates computation and hence, the performance of all four scheduling algorithms tends to degrade. However, as shown in Fig. 4(a) and (b), the HEFD algorithm is more effectively able to deal with the increase in communication cost compared to HEFT, HLD, and HCPFD algorithms. The ability of the HEFD algorithm to efficiently handle the increase in communication overhead can be explained as follows. As the CCR value increases, the weight of task nodes and edges can be efficiently handled by Eqs. (4) and (5). Hence, heavily communicating tasks will be identified and selected for scheduling before other tasks with duplication. Moreover, at each scheduling step, the insertion-based scheduling policy assigns the selected task to a processor that minimizes its execution finishing time. Thus, more heavily communicating immediate parent tasks will be selected and assigned to the same processor if such an assignment leads to a shorter provisional schedule.

The results of the effect of computation heterogeneity on the performance of algorithms are shown in Fig. 5. On an average performance is found to deteriorate with increasing heterogeneity. This observation reflects that any misjudgment or lack of farsightedness on the part of heuristic may prove more costly in HCS, as the task may get scheduled on a less suitable processor (unlike homogeneous system where all the processors are identical). Weight of

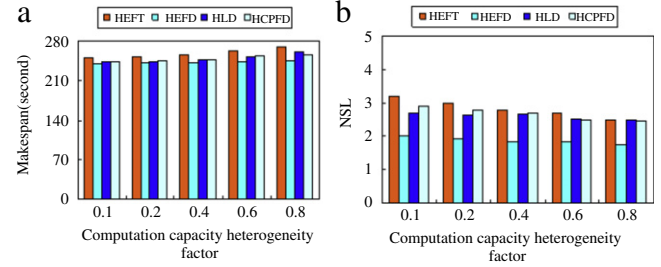


Fig. 5. Performance impact of computation capacity heterogeneity factor α : (a) makespan in seconds; (b) NSL.

task nodes and edges based on computation and communication heterogeneity, however, succeeds to some extent in overcoming these injudicious moves. In comparison to HEFT, HLD, and HCPFD algorithms, average makespan generated by HEFD algorithm is better by as much as 7.6%, 3.4%, and 4.0%, respectively, average NSL is also better by as much as 25.3%, 19.4%, and 18.6%, respectively.

5.4. Performance analysis on application graphs of real-world problems

Using real applications to test the performance of algorithms is very common [21,32,33,6]. Hence, in addition to randomly generated DAGs, we also simulate a digital signal processing (DSP) example [14,33].

We select a DSP example to test HEFD algorithm because the computation time and the communication data can be estimated very accurately. There are 119 tasks in the DSP task graph. The task graph of the DSP and the parameters for the DSP can be found in [21,33]. In this case, we just change the computation capacity heterogeneity factor α of HCS, and five different values: 0.1, 0.2, 0.4, 0.6, and 0.8. We selected the appropriate weighting parallelism factor λ and ran the algorithm 500 times in each case. The results are shown in Fig. 6.

Fig. 6 shows that when the computation capacity heterogeneity factor α is small, the performance improvement of HEFD algorithm is inconspicuous. This phenomena confirm that our new weight assigning method cannot improve the performance of homogeneous system. As α increase, the performance of HEFD algorithm is also increase. The makespan of HEFD is shorter than HEFT, HLD and HCPFD by 18.3%, 9.8% and 7.8%, respectively. And the NSL of HEFD algorithm outperforms HEFT, HLD and HCPFD by 8.3%, 4.2% and 3.8%, respectively. That shows HEFD algorithm is very suitable for HCS with higher heterogeneity.

6. Conclusion

We present a new algorithm which incorporating duplication-based techniques into list-based scheduling, called heterogeneous

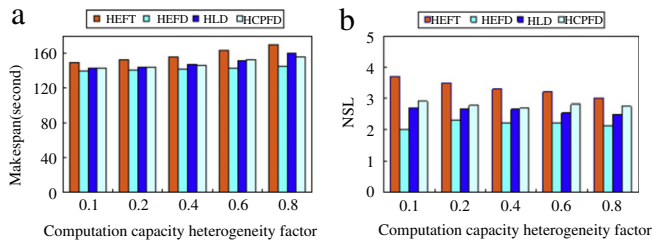


Fig. 6. Experiment results of DSP: (a) makespan in seconds; (b) NSL.

earliest finish with duplicator (HEFD), for the problem of scheduling in heterogeneous computing systems (HCS). The HEFD algorithm uses a new weight of task nodes and edges to accurately identify the priorities of tasks in HCS. To reduce task start execution time, straightforward duplication approach is proposed, and try to exploit all immediate parents to get optimize results.

The performance of the HEFD algorithm is compared to three of the best existing scheduling algorithms for HCS: the HEFT, HLD, and HCPFD algorithms. The comparative study is based on both randomly generated application DAGs and DAGs that correspond to real-world numerical applications. The HEFD algorithm outperforms HEFT, HLD, and HCPFD algorithms in terms of makespan and normalized schedule length (NSL).

This work represents our first and preliminary attempt to study a very complicated problem. Future studies in this area are twofold. First, the involvement of weight of task nodes and edges should be further investigated in order to improve task scheduling accuracy and efficiency. Second, we plan to extend HEFD algorithm to the dynamic environment where processor load, capability, and network condition vary during the execution of workflow applications.

Acknowledgments

The authors would like to thank the anonymous reviewers for their valuable comments and suggestions. This work was partially supported by the National Science Foundation of China under grant nos. 90715029 and 60673061, a project supported by the Cultivation Fund of the Key Scientific and Technical Innovation Project, Ministry of Education of China (Grant No. 708066), a project supported by Scientific Research Fund of Hunan Provincial Education Department (08C435, 09A043).

References

- [1] I. Ahmad, Y.-K. Kwok, On exploiting task duplication in parallel program scheduling, *IEEE Trans. Parallel Distrib. Systems* 9 (9) (1998) 872–892.
- [2] I. Ahmad, Y. Kwok, A comparison of task-duplication-based algorithms for scheduling parallel programs to message-passing systems, in: *Proc. 11th Int. Symp. of High-Performance Computing Systems*, 1997, pp. 39–50.
- [3] S. Bansal, P. Kumar, K. Singh, An improved duplication strategy for scheduling precedence constrained graphs in multiprocessor systems, *IEEE Trans. Parallel Distrib. Systems* 14 (6) (2003) 533–544.
- [4] Savina Bansal, Padam Kumar, Kuldeep Singh, Dealing with heterogeneity through limited duplication for scheduling precedence constrained task graphs, *J. Parallel Distrib. Comput.* 65 (4) (2005) 479–491.
- [5] H. Casanova, Network modeling issues for grid application scheduling, *Int. J. Found. Comput. Sci.* 16 (2) (2005) 145–162.
- [6] Y. Chung, S. Ranka, Application and performance analysis of a compile-time optimization approach for list scheduling algorithms on distributed memory multiprocessors, in: *Proc. Super Computing*, 1992, pp. 512–521.
- [7] Mohammad I. Daoud, Nawwaf Kharma, A high performance algorithm for static task scheduling in heterogeneous distributed computing systems, *J. Parallel Distrib. Comput.* 68 (4) (2008) 399–409.
- [8] S. Darbha, D.P. Agrawal, Optimal scheduling algorithm for distributed-memory machines, *IEEE Trans. Parallel Distrib. Systems* 9 (1) (1998) 87–95.
- [9] S. Darbha, D. Agrawal, A fast and scalable scheduling algorithm for distributed memory systems, in: *Proc. of Symp. on Parallel and Distributed Processing*, 1995, pp. 60–63.

- [10] M.K. Dhodhi, I. Ahmad, A. Yatama, et al., An integrated technique for task matching and scheduling onto distributed heterogeneous computing system, *J. Parallel Distrib. Comput.* 62 (9) (2002) 1338–1361.
- [11] A. Dogan, F. Özgüner, LDBS: a duplication based scheduling algorithm for heterogeneous computing systems, in: *Proceedings of the International Conference on Parallel Processing, ICPP-2002, Canada, 2002*, pp. 352–359.
- [12] H. El-Rewini, T.G. Lewis, Scheduling parallel program tasks onto arbitrary target machines, *J. Parallel Distrib. Comput.* 9 (2) (1990) 138–153.
- [13] H. El-Rewini, T.G. Lewis, H.H. Ali, *Task Scheduling in Parallel and Distributed Systems*, Prentice Hall, Englewood Cliffs, NJ, 1994.
- [14] M.R. Garey, D.S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W.H. Freeman and Co, San Francisco, 1979.
- [15] T. Hagras, J. Janacek, A high performance, low complexity algorithm for compile-time task scheduling in heterogeneous systems, *Parallel Comput.* 31 (7) (2005) 653–670.
- [16] M. Iversen, F. Ozuner, G. Follen, Parallelizing existing applications in a distributed heterogeneous environment, in: *Proceedings of Heterogeneous Computing Workshop*, 1995, pp. 93–100.
- [17] D. Kim, B.G. Yi, A two-pass scheduling algorithm for parallel programs, *Parallel Comput.* 20 (6) (1994) 869–885.
- [18] Y.-K. Kwok, I. Ahmad, Static scheduling algorithms for allocating directed task graphs to multiprocessors, *ACM Comput. Surv.* 31 (4) (1999) 406–471.
- [19] Y.-K. Kwok, I. Ahmad, Benchmarking and comparison of the task graph scheduling algorithms, *J. Parallel Distrib. Comput.* 59 (3) (1999) 381–422.
- [20] Y.-K. Kwok, I. Ahmad, Dynamic critical-path scheduling: An effective technique for allocating task graphs onto multiprocessors, *IEEE Trans. Parallel Distrib. Systems* 7 (5) (1996) 506–521.
- [21] G.Q. Liu, K.L. Poh, M. Xie, Iterative list scheduling for heterogeneous computing, *J. Parallel Distrib. Comput.* 65 (5) (2005) 654–665.
- [22] C.H. Papadimitriou, M. Yannakakis, Towards an architecture-independent analysis of parallel algorithms, *SIAM J. Comput.* 19 (2) (1990) 322–328.
- [23] H.J. Park, B.K. Kim, An optimal scheduling algorithm for minimizing the computing period of cyclic synchronous tasks on multiprocessors, *J. Syst. Softw.* 56 (3) (2001) 213–229.
- [24] Larry L. Peterson, Bruce S. Davie, *Computer Networks: A Systems Approach*, third ed., Morgan Kaufmann Publishers, Los Altos, CA, 2003.
- [25] A. Radulescu, A.J.C. van Gemund, Low-cost task scheduling for distributed-memory machines, *IEEE Trans. Parallel Distrib. Systems* 13 (6) (2002) 648–658.
- [26] Zhao Shi, Jack J. Dongarra, Scheduling workflow applications on processors with different capabilities, *Future Gener. Comput. Syst.* 22 (6) (2006) 665–675.
- [27] G.C. Sih, E.A. Lee, A compile-time scheduling heuristic for interconnection-constrained heterogeneous machine architectures, *IEEE Trans. Parallel Distrib. Systems* 4 (2) (1993) 175–187.
- [28] O. Sinnen, L.A. Sousa, Communication contention in task scheduling, *IEEE Trans. Parallel Distrib. Systems* 16 (6) (2005) 503–515.
- [29] O. Sinnen, L.A. Sousa, List scheduling: extension for contention awareness and evaluation of node priorities for heterogeneous cluster architectures, *Parallel Comput.* 30 (1) (2004) 81–101.
- [30] Xiaoyong Tang, Li Kenli, D. PADUA, Communication contention in APN list scheduling algorithm, *Science in China Series F: Information Sciences* 52 (1) (2009) 59–69.
- [31] H. Topcuoglu, S. Hariri, M.Y. Wu, Task scheduling algorithms for heterogeneous machines, in: *Proceedings of the Heterogeneous Computing Workshop*, Mexico, 1999, pp. 3–14.
- [32] H. Topcuoglu, S. Hariri, M.-Y. Wu, Performance-effective and low-complexity task scheduling for heterogeneous computing, *IEEE Trans. Parallel Distrib. Systems* 13 (3) (2002) 260–274.
- [33] C.M. Woodside, G.G. Monforton, Fast allocation of processes in distributed and parallel systems, *IEEE Trans. Parallel Distrib. Systems* 4 (2) (1993) 164–174.
- [34] H. Zhao, R. Sakellariou, An experimental investigation into the rank function of the heterogeneous earliest finish time scheduling algorithm, in: *Proceedings of 9th International Euro-Par Conference*, Springer-Verlag, 2003, pp. 189–194.



Xiaoyong Tang received his master degree from Hunan University, China, in 2007. He is currently working towards the Ph.D. degree at Hunan University of China. His research interests include modeling and scheduling for distributed computing systems, distributed system reliability, distributed system security, and parallel algorithms.



Kenli Li received the Ph.D degree in computer science from Huazhong University of Science and Technology, China, in 2003, and the B.S. degree in mathematics from Central South University, China, in 2000. He has been a visiting scholar at University of Illinois at Champaign and Urbana from 2004 to 2005. He is now a professor of Computer science and Technology at Hunan University. He is a senior member of CCF. His major research include parallel computing, Grid and Cloud computing, and DNA computer.



Guiping Liao received the M.S. degree in ecology from Institute of Applied Ecology Research, Chinese Academy of Sciences, China, in 1994, and Ph.D. degree in agronomy from Hunan Agricultural University, China, in 2000. He was a Post-Doctor of School of Computer, National University of Defense technology, China, from November, 2004 to November, 2005. He was a visiting scientist of the Plant Biotechnology Institute (PBI) of National Research Council, Canada, from November, 2004 to November, 2005. He is currently a director of Institute of Agricultural Information technology, and a professor and Ph.D. supervisor of School of Information Science and Technology, Hunan Agricultural University. His research interests include expert system,

intelligent information processing, distributed computing systems, and computer vision.



Renfa Li received the Ph.D. degree in computer science from Huazhong University of Science and Technology, China, in 2001. He is a professor and Ph.D. supervisor. He is also a senior member of China Computer Federation. His main research interests include embedded computing, distributed computing systems, and wireless network.